

Teste Técnico – Fase 2 (Nova Itapemirim)

Contexto

O desafio consiste em desenvolver um sistema de gerenciamento de produtos utilizando **Angular**, integrado à API **fakestoreapi**. Diferente de um CRUD básico, esperamos que a entrega demonstre maturidade em **arquitetura, decisões técnicas e qualidade de código** – os mesmos fundamentos exigidos no dia a dia do projeto em que você atuará.

- Documentação da API: [fakestoreapi documentation](https://fakestoreapi.com/)

Tarefa

Criar um projeto Angular do zero (Angular 17 ou superior) contendo:

1. Autenticação

- Tela de login integrada ao endpoint `POST /auth/login` da fakestoreapi (as credenciais de teste estão na documentação da API);
- Armazenamento e gestão do token de sessão;
- Um **HTTP Interceptor** que injete o token nas requisições autenticadas;
- Um **Route Guard** que proteja as rotas do CRUD – usuário não autenticado deve ser redirecionado ao login;
- Tratamento de expiração/invalidadez de sessão (logout e redirecionamento).

2. CRUD de Produtos

- Criar um novo produto (nome, preço, descrição, categoria, imagem);
- Listar todos os produtos disponíveis na API;
- Atualizar as informações de um produto existente;
- Excluir um produto da lista (com confirmação antes da exclusão).

Nota: a fakestoreapi não persiste escritas (POST/PUT/DELETE retornam sucesso, mas não alteram os dados). Esperamos que você identifique isso e trate no front (ex.: refletir as mutações no estado local da aplicação), documentando a decisão no README.

3. Listagem – critérios de aceitação

- Paginação;
- Busca com filtros por **Título** (parcial), **Preço** (mínimo e máximo) e **Categoria**;
- A busca por título deve usar **debounce** (evitar processamento/requisição a cada tecla);

- Estados de interface tratados: **carregando** (skeleton ou spinner), **lista vazia**, **erro de requisição** (com possibilidade de retry).

Requisitos de Arquitetura e Código

Estes pontos têm peso alto na avaliação:

1. **Organização do projeto** em camadas claras (ex.: `core/` , `shared/` , `features|pages/`), com separação entre serviços de acesso à API, modelos e componentes de apresentação;
2. **Lazy loading** nas rotas de feature;
3. **Layout responsivo é obrigatório**: a aplicação deve funcionar bem em desktop e mobile;
4. **Tipagem forte de ponta a ponta**: modelos de request e response da API tipados (sem `any`); tipar também erros de HTTP;
5. **Boas práticas de performance**: `ChangeDetectionStrategy.OnPush` onde fizer sentido, `trackBy` em listas, cancelamento/troca adequada de requisições concorrentes (ex.: `switchMap` na busca), prevenção de memory leaks (`unsubscribe` / `takeUntil` / `async pipe`);
6. **Tratamento centralizado de erros HTTP** (interceptor ou handler dedicado) com feedback ao usuário (toast/alert);
7. **Componentização**: ao menos os componentes de paginação e filtros devem ser reutilizáveis e desacoplados da feature de produtos.

Diferenciais (não obrigatórios)

- Desenvolver o projeto com **ionic** (Ionic + Angular);
- **Testes unitários** com a biblioteca de sua preferência (Karma/Jasmine, Jest, etc.), cobrindo pontos relevantes como o serviço de produtos, o guard de autenticação ou componentes com lógica – valorizamos testes que verifiquem comportamento de verdade, não apenas `should create` ;
- `Dockerfile` com build multi-stage (build Angular + nginx) para servir a aplicação;
- Configuração de múltiplos ambientes (`environment.ts` / `environment.prod.ts`) com a URL da API parametrizada;
- Cuidado com acessibilidade (semântica HTML, navegação por teclado, atributos ARIA);
- Cuidado com o layout e a experiência do usuário.

Entrega

- Prazo: até **5 dias úteis** após o recebimento da tarefa;
- Código disponibilizado em repositório no **GitHub** com permissão de acesso para avaliação;
- **README** obrigatório contendo:

- Instruções de execução do projeto;
- Visão geral da arquitetura e organização de pastas;
- **Decisões técnicas e trade-offs** — esta seção é parte da avaliação: explique o porquê das suas escolhas (estrutura, gerenciamento de estado, bibliotecas, tratamento do comportamento da API, etc.);
- Bibliotecas de terceiros utilizadas e o motivo de cada uma.

Avaliação

A entrega será avaliada por revisão de código e discutida em uma **conversa técnica**, na qual pediremos que você explique e defenda suas decisões. Os principais critérios, em ordem de relevância:

1. Arquitetura e organização do código;
2. Domínio de boas práticas do Angular;
3. Tratamento de erros e estados de interface;
4. Clareza do README e das decisões documentadas;
5. Layout, responsividade e experiência do usuário.

Observações

- É permitido utilizar bibliotecas adicionais (documente o que foi utilizado e por quê);
- O tempo de entrega dentro do prazo não será fator de avaliação. Nosso foco está na **qualidade da entrega**.